

DA5300: Data Structures For Data Science

Laboratory 5 - General Trees and Binary Trees

Objectives

Implement general and binary trees in Python. Use file I/O, regular expressions, tree nodes, parent-child relationships, and pre-order/post-order traversals.

Part A: General Tree – Representing a Book

A book consists of chapters, sections, subsections, sub-subsections, and so on. Chapters are numbered as 1., 2., etc.; sections as 1.1, 1.2, 2.1, etc.; and further levels as 2.1.1, 2.1.1.1, etc. The book title does not begin with a number.

The input file contains these lines in *random order*. A finest-level entry contains its page count:

```
2.4.2.1 Recursion : 7
```

The number of components in the label determines its depth. The parent of 2.4.2.1 is 2.4.2; the parent of 2.4.2 is 2.4, and so on.

A1. Node

Create a `Node` class containing a title, its number (if any), a list of children, and a page count where applicable.

A2. Construct the General Tree

Read `book_titles.txt`. Use a regular expression to separate the hierarchical number from the title and, where present, the page count. Construct a general tree rooted at the book title. Do not assume that the input lines are in hierarchical order.

A3. Post-order Page Calculation

Implement post-order traversal. A finest-level node has the page count given in the input. The page count of every internal node is the sum of the page counts of its children. Thus calculate the page count of every chapter, section, and the complete book.

A4. Pre-order Contents

Implement pre-order traversal and generate the table of contents in `book_content.txt`. Print each chapter/section/subsection in hierarchical order together with its starting page number. The starting page number of a node is obtained using the page spans computed from the post-order calculation.

A5. Command Line

The program must be named `book.py`. Run it as:

```
python book.py book_titles.txt book_content.txt
```

Do not hard-code the filenames (pass them as arguments from command line).

Part B: Binary Tree from Traversals

The first line of `prein.txt` contains a pre-order traversal and the second line contains an in-order traversal. Construct the binary tree and produce its post-order traversal.

B1. Node

Create a binary-tree node containing an element, a left-child reference and a right-child reference.

B2. Construct the Tree

Use the fact that the first element of pre-order is the root. Locate it in the in-order traversal; the elements to its left form the left subtree and those to its right form the right subtree. Recursively construct the complete tree.

B3. Post-order

Implement post-order traversal and write the result to `post.txt`.

B4. Command Line

The program must be named `order.py`. Run:

```
python order.py prein.txt post.txt
```

Part C: Test Cases

For each test case, create the specified input file, run the program, and compare its output with the expected result.

C1. General Tree Test Cases

Test 1

```
Data Structures
2. Trees : 3
1. Arrays
2.1 General Trees : 5
1.1 Array Basics : 4
2.2 Binary Trees : 6
1.2 Array Operations : 7
```

Test 2

```
Algorithms
3. Graph Algorithms
1. Searching
2. Sorting
3.2 Shortest Paths : 8
1.1 Linear Search : 4
2.1 Bubble Sort : 5
3.1 Graph Representation : 6
3.3 Minimum Spanning Trees : 9
1.2 Binary Search : 6
2.2 Merge Sort : 7
```

Test 3

```
Introduction to Computer Science
3. Programming
1. Foundations
4. Data Structures
2. Algorithms
3.2 Object Oriented Programming : 8
4.1 Trees : 10
1.1 Variables and Types : 5
2.1 Searching : 6
3.1 Functions : 4
4.2 Graphs : 12
1.2 Control Flow : 7
2.2 Sorting : 9
3.3 File Handling : 5
```

C2. Binary Tree Test Cases

Test 1

```
A B D E C F
D B E A C F
```

Test 2

```
A B D C E F
D B A E C F
```

Test 3

```
A B C D E F G
A C B D E G F
```

Part D: Evaluation

Two evaluator programs are supplied.

For the book:

```
python book_evaluate.py book_titles.txt book_content.txt
```

For the binary tree:

```
python order_evaluate.py prein.txt post.txt
```

The evaluators should be run after your programs have generated the required output files.